

Multi Agent Path Planning using Ant Colony Optimisation

蟻コロニー最適化方法を用いたマルチエージェント経路探索

18KMH18 Joly Raphaël

研究指導教員 教授 五十嵐 洋

Abstract - Multi-Agent Path Planning (MAPP) is a NP-hard problem which finds its application in logistic or in game development for example. The idea of this research is to use Ant Colony Optimisation (ACO) to solve a MAPP problem in a nodes and connections graph environment with narrow connections. For this, two types of algorithm are proposed, one is search-based and the other rule-based and the search-based one is evaluated in term of computation time and success rate.

I Introduction

1.1 Motivation

Multi-agent path planning is a challenging problem. MAPP is adding other difficulties compared to a single agent path planning such as collision between agents and congestion. As collision and congestion are difficult to detect and anticipate, usually MAPP complexity is exponential considering the number of agents.

Also, MAPP algorithm efficiency can greatly change depending on the type of environment, that's why it is important to adapt the algorithm to the environment it is used on. The idea of this research is to devote the detection and anticipation of collision to a Swarm Intelligence algorithm and to combine its results with an existing optimal path planning for single agent.

There are two motivations for the choice of Ant Colony Optimisation. The first one is that the use of an AI for complex problem are good methods that trades optimality for efficiency compared to analytical methods. The second one is that the object named pheromone map created through the ACO algorithm allow for a simplified path recalculation in case of an unexpected event.

1.2 Related works

MAPP algorithm efficiency is strongly related to the environment. That's why previous works on MAPP problems often focus on a single scenario

such as 2-dimensionnal discrete space division in [7] for example. Swarm intelligence has also been the AI choice of some other research such as [3] where it's used to create a distribution of vehicles on a real map. Existing methods for MAPP are search-based, reduction-based or rule-based [6]. Search-based methods will calculate agent's paths one at a time, any agent searching for a path may not occupy specific locations at specific times reserved by previous agents. Rules-based methods consist on imposing rule of movement for every agent based on pattern on the map or any other circumstances. An example can be seen in [4] where each agent has its own movement rules depending on previous environment states. Reduction-based methods consist on reducing the MAPP problem to a standard known problem such as the Boolean satisfiability problem (SAT) for which the solvability and efficiency have already being proven.

1.3 Contribution

In industrial workplace, the method using proximity sensing to avoid collision are usually prioritized. If those methods are effective in a vast environment with few agents, it fails to avoid congestions in cases where paths are narrow or when there is a high amount of traveling agents. This is why this research seeks to adapts to scenarios with restricted space such as warehouses travelled by transport machines or fully automated industrial places. The better distribution of agents on the map allowed by MAPP methods combined with a proximity detection should create an optimal collision avoidance algorithm for all sort of applications implying many automated traveling vehicles.

II Restrictions and Problem Formulation

2.1 Restrictions

This research is focusing on a 2-dimensional graph divided in nodes and edges. Every nodes have a coordinate and edges have a positive distance. Concerning the way vehicles are traveling on the map,

this research is considering the case where an edge is large enough to only let pass a single vehicle. It means that the following cases are considered as collision:

- Vehicles taking the same edge on opposite direction
- Two vehicles crossing a node at the same time while taking opposite direction. Indeed, if both vehicles don't take the opposite direction, a crossing solution where one vehicle is waiting while the other one is crossing can be found.
- Vehicles entering the same edge at the same period of time on the same direction. Period of time is defined according to a minimum security distance between vehicles.

For simplicity, this research assume that all vehicles have a constant speed while moving around the map and have the same behaviour when they stop, accelerate or turn.

2.2 Problem Formulation

The main task for a vehicle is to go from a node of the map to another. The goal of the algorithm is to avoid any collision between vehicles on the map and to return a "fail" state if collision cannot be avoided (case where there's too much vehicles on the map for example).

III Algorithm

When the path of a group of vehicles must be calculated, the algorithm runs as in Fig. 1.

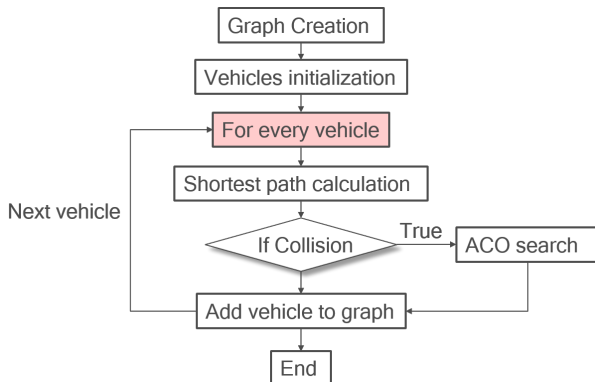


Fig. 1 MAPP determination's flow chart

3.1 Single Agent Trajectory Generation

This research is using an already optimized single agent path planning algorithm to function called Dijkstra algorithm. Dijkstra algorithm has been proved to be very efficient for optimal pathfinding

in real map and in other various areas.

As the main difficulty of the an AI is that an adaptation time is required before converging to a solution. In the case of the Ant Colony Optimisation (ACO), the initial search is performed randomly, without real knowledge on the map. By using the Dijkstra algorithm calculated path, the ACO will prioritise path around the single agent optimised path which are usually the fastest ones. Also, for the same purpose, the ACO calculation is performed on a simplified map where dead ends are eliminated.

3.2 Multi Agent Algorithm

ACO is a swarm intelligence algorithm that imitates the behavior of real ants. The algorithm is based on a new type of variable, the pheromone, that will be use as a weight coefficient throughout the algorithm's search^[2]. The ACO algorithm, adapted to the nodes and edges map, is the following:

- A wave of "ants" is searching the space until it reaches the goal.
- At every nodes, an ant can choose an edge with a certain probability. This probability depends on the amount of pheromone of every edges of the node and also on an "road affinity" function decided beforehand.
- When every ants of the wave has reach the objective or after a certain amount of steps, pheromones are added to the ants' route depending on the efficiency of this one. Usually, the shorter the path is, the higher is the amount of pheromone the path will receive.
- After every wave, a pheromone dissipation is performed on the whole map to eliminate the least used paths.

To take into account the other vehicles during the ACO search, the pheromone weight of each edges is depending on other vehicles' path and position. Therefore the pheromone weight on each edge is a function that depends on time.

To gather and process the pheromone's data, I use an object called "Pheromone map". This object contains the value of pheromone of every edge at any time of the simulation. An example of pheromone map can be seen in Fig. 2. On this example, the colour of the arrows corresponds to the weight of pheromone on it. It can be noticed that, on an edge, one direction and its opposite are considered differently. Also, as it has been explained in ^[1], I defined

a maximum and minimum value for the pheromone. I ensured that for any situation, the minimum probability for an ant to take a path is 5%.

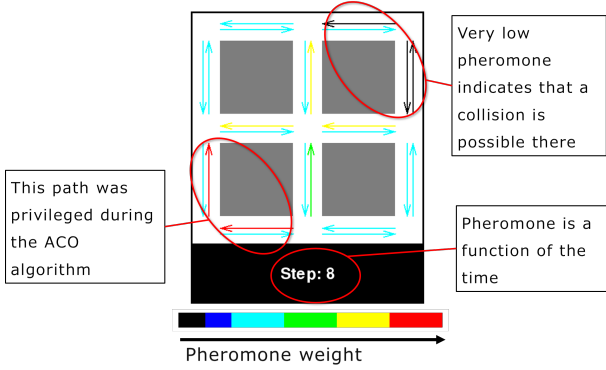


Fig. 2 Example of pheromone map

One important point in my algorithm is the fact that the ACO search is only used when a collision is detected with the vehicles already added to the graph. It also means that the order in which the vehicles are added has an importance on the final path length. My approach is a search-based method as I am using the results of other agents to calculate a new path. However, we will see that calculating the path of agents one by one poses a problem of mutual optimality as the first agent for which the path is calculated will usually have a shorter distance to travel compared to the later agents. That's why I also considered using a rule-based approach in the development of my algorithm.

3.3 Rule-based approach

The rule-based algorithm approach is to calculate the path of all the agents at the same time. The agents next movement is imposed by a list of rules sorted by importance. The methods for a MAPP determination is as follow:

- At the start, every agents is choosing a path considering the neighbours agent.
- Step by step, the agents advance and the pheromones are updated to prevent the other agents to go the opposite sides.
- When an agent reach an intersection, it can either: cross the intersection if no other agent is there or stop to let an other agent cross.
- If two or more agent arrives at the same intersection, a passage order is decided.
- If the intersection is blocked by too many agents, one of them do a U-turn and a passage order is decided for the remaining agents.

This algorithm is expected to have a better success rate than the search-based algorithm because it calculate the paths of the agents altogether (See evaluation section on why this is required). Also, it would also improve average traveling time of all the agents as the pheromone would be updated considering the overall travel time instead of the solo travel time like in the search-based algorithm.

IV Evaluation

To evaluate this algorithm, several experiences were conducted, and the computational time and success rate were fetched. During these experiences, the graph's size and the number of agents were changed and a total of 100 tests were conducted for each set of parameters. To get a graph of the wanted size, a random graph generator has been used. You can see an example of a 20 nodes randomly generated graph on the Fig. 3.

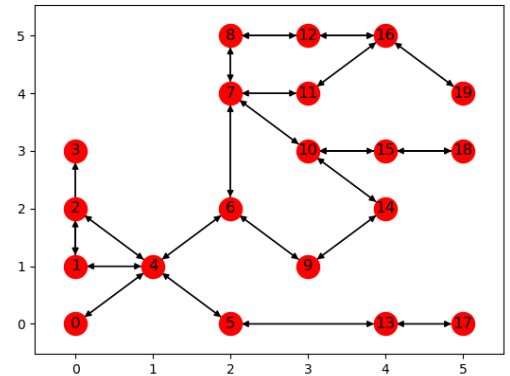


Fig. 3 Randomly generated 20 nodes graph

The results for the computational time are displayed on Fig. 4. The computational time seems to increase linearly as the number of agents increases. This means that it should be possible with optimisation and a dedicated system to get an average good computational time for this algorithm even with a high number of agents.

For these tests, success rate is defined as: “Is there still a collision after calculation?” . In other word, was the algorithm able to give a proper non-collision path for all the agents on the map. The results for the success rate are displayed on Fig. 5.

On these results, it can be noticed that there is a large drop in success rate as the number of agents increases. These can be caused by two factors, either the saturation of the map which correspond to unsolvable scenario with the group of agents. And

also the fact that the path of the agents is calculated one after the other which can also lead to unsolvable scenario for the last agents as shown in Fig. 6.

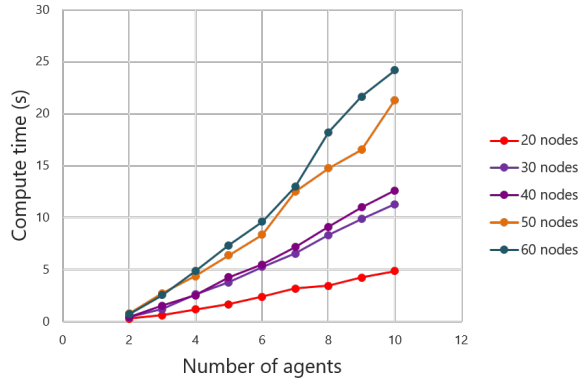


Fig. 4 Computation time considering the number of vehicles and graph's size

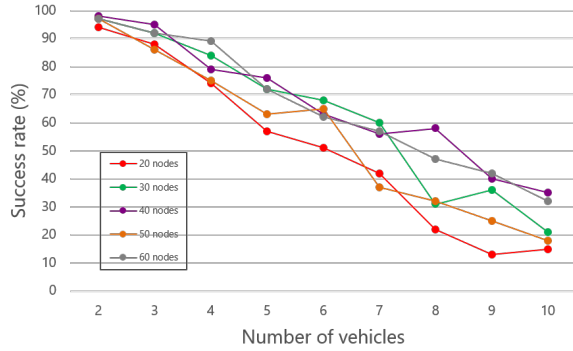


Fig. 5 Success rate considering the number of vehicles and graph's size

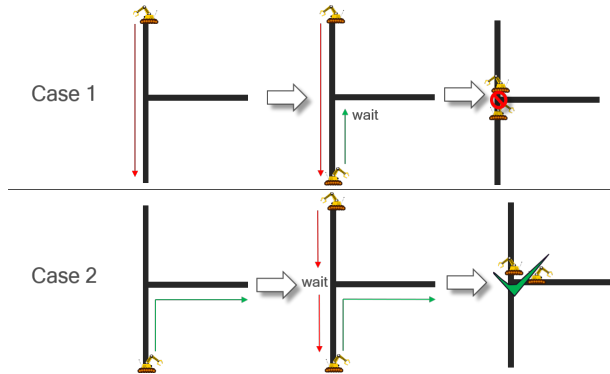


Fig. 6 Unsolvable scenario induced by calculation order

V Conclusion

This research achieved to create a MAPP algorithm using ACO for graph environment with narrow connections restrictions. The developed algorithm is search based and is using the ACO's pheromones to guide every agent through the map without colliding with the other. However, the evaluation of these algorithm shows a low success rate mostly due to the unsolvable scenario induced by the search-based type of algorithm used. This is why a

rule-based algorithm have also been proposed in this research. The future work for this research would be to implement the rule-based MAPP using ACO and compare its result with the search-based algorithm, especially on success rate.

References

- [1] Stützle, Thomas, and Holger H. Hoos. "MAX-MIN ant system." *Future generation computer systems* 16.8 (2000): 889-914.
- [2] Dorigo, Marco, and Mauro Birattari. *Ant colony optimization*. Springer US, 2010.
- [3] Stolcis, Christian, and Elmar Pfannerstill. "Clustered swarm: a live swarm-based traffic load balancing algorithm against traffic jams." *IET Intelligent Transport Systems* 11.3 (2016): 134-141.
- [4] Li, Howard, Simon X. Yang, and Mae L. Seto. "Neural-network-based path planning for a multirobot system with moving obstacles." *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 39.4 (2009): 410-419.
- [5] Hönig, Wolfgang, et al. "Persistent and robust execution of mapf schedules in warehouses." *IEEE Robotics and Automation Letters* 4.2 (2019): 1125-1131.
- [6] Felner, Ariel, et al. "Search-based optimal solvers for the multi-agent pathfinding problem: Summary and challenges." *Tenth Annual Symposium on Combinatorial Search*. 2017.
- [7] Robinson, D. Reed, et al. "An efficient algorithm for optimal trajectory generation for heterogeneous multi-agent systems in non-convex environments." *IEEE Robotics and Automation Letters* 3.2 (2018): 1215-1222.